

Inteligência Computacional

Desafio 1: Mapa de Características

João Geiger Piza

Ruan Pablo de Lima Coelho

Para definir a função phi, primeiramente transformamos os dados da entrada no tipo Double para maior precisão.

```
def phi(self, X: torch.Tensor) -> torch.Tensor:
    X_duplo = X.double()
    Xs = self._padronizar(X_duplo)

    explicitas = self._extrair_caracteristicas_explicitas(Xs)
```

Após isso normalizamos os dados com a função padronizar.

```
def _padronizar(self, X_duplo: torch.Tensor) -> torch.Tensor:
    return (X_duplo - self.media) / self.desvio
```

Essa função padronizar aplica a padronização z-score, que é definida pela fórmula: $z = \frac{X - \mu}{\sigma}$

Com os dados padronizados, calculamos as “características explícitas”

```
def _extrair_caracteristicas_explicitas(self, Xs: torch.Tensor) -> torch.Tensor:
    d = Xs.shape[1]
    raio_quadrado = (Xs ** 2).sum(dim=1, keepdim=True)
    caracteristicas = [Xs, Xs ** 2, raio_quadrado]

    if d == 2:
        produto_cruzado = Xs[:, 0:1] * Xs[:, 1:2]
        angulo_theta = torch.atan2(Xs[:, 1:2], Xs[:, 0:1])
        caracteristicas.extend([
            produto_cruzado,
            torch.sin(angulo_theta),
            torch.cos(angulo_theta),
            torch.sin(2 * angulo_theta),
            torch.cos(2 * angulo_theta),
            torch.sin(3 * angulo_theta),
            torch.cos(3 * angulo_theta)
        ])

    return torch.cat(caracteristicas, dim=1)
```

Que são as características polinomiais, radiais e harmônicas em coordenadas polares do tensor de dados inicial. Algumas delas são exclusivas para problemas bidimensionais.

Com o número de características explícitas calculado, podemos completar o tensor resultante com características geradas por meio de random fourier features, para assim ao final chegarmos em um tensor com 64 dimensões(máximo permitido pelo desafio)

```
num_fourier = self.DIM_MAX - explicitas.shape[1]

fourier = self._gerar_caracteristicas_fourier(Xs, num_fourier)

resultado = torch.cat([explicitas, fourier], dim=1)
return resultado.float()
```

Para gerar as características de fourier iremos multiplicar os dados iniciais com a matriz de projeção W e somar com o vetor de fase b .

```
def _gerar_caracteristicas_fourier(self, Xs: torch.Tensor, num_fourier: int) -> torch.Tensor:
    """Aplica a projeção de Fourier Aleatória (RFF) multi-escala."""
    projecao = Xs @ self.matriz_W[:, :num_fourier] + self.vetor_b[:num_fourier]
    return math.sqrt(2.0) * torch.cos(projecao)
```

Por fim aplicamos a transformação não linear $\sqrt{2} * \cos(XW+b)$ para obter a representação final

Para criar as várias random furrier features, geramos uma matriz de pesos de Fourier em múltiplas escalas (com fatores variando de 0.25 a 3.0 sobre o kernel base).

```
def _gerar_pesos_fourier(self, d: int, num_fourier: int, sigma_base: float) -> None:
    """Gera a matriz W e o vetor b com gerador determinístico em múltiplas escalas."""
    gerador = torch.Generator().manual_seed(2026)
    escalas = [0.25, 0.5, 1.0, 1.5, 2.0, 3.0]
    colunas_por_escala = math.ceil(num_fourier / len(escalas))

    blocos_W = []
    for escala in escalas:
        sigma_atual = sigma_base / escala
        w_escala = torch.randn(d, colunas_por_escala, generator=gerador, dtype=torch.float64) / sigma_atual
        blocos_W.append(w_escala)

    self.matriz_W = torch.cat(blocos_W, dim=1)[: , :num_fourier]
    self.vetor_b = torch.rand(num_fourier, generator=gerador, dtype=torch.float64) * 2 * math.pi
```

Iremos calibrar as RFF de maneira adaptativa ao conjunto de dados, estimando a escala base do kernel usando a heurística da mediana sobre as distâncias quadradas entre uma amostragem dos pontos.

```
def _estimar_sigma_base(self, Xs: torch.Tensor) -> float:
    """Estima a distância mediana entre os pontos para calibrar a escala gaussiana (sigma)."""
    indices = torch.randperm(len(Xs))[:300]
    amostra = Xs[indices]
    distancias_quadradas = torch.cdist(amostra, amostra) ** 2
    valores_positivos = distancias_quadradas[distancias_quadradas > 0]
    mediana = valores_positivos.median().item() if len(valores_positivos) > 0 else 1.0
    return math.sqrt(mediana / 2.0) + 1e-8 # evitar div por 0
```

Para definir a função fit, também transformamos os dados da entrada no tipo Double para maior precisão.

```
def fit(self, X: torch.Tensor) -> None:
    X_duplo = X.double()
    self.media = X_duplo.mean(dim=0)
    self.desvio = X_duplo.std(dim=0) + 1e-8

    Xs = self._padronizar(X_duplo)
    d = X.shape[1]
```

E padronizamos os dados com a função padroniza.

Em seguida calculamos o número de características explícitas e subtraímos do valor limite da dimensão do tensor final, assim chegando ao valor total de random fourier features que serão calculadas.

```
explicitas_m = self._extrair_caracteristicas_explicitas(Xs[:1])  
num_explicitas = explicitas_m.shape[1]  
num_fourier = self.DIM_MAX - num_explicitas  
  
sigma_base = self._estimar_sigma_base(Xs)  
self._gerar_pesos_fourier(d, num_fourier, sigma_base)
```

Gerando, por fim, a matriz pesos de Fourier calibrada para completar a dimensão total.